

Capítulo 3

Vacío de la intersección de una RCG con un lenguaje regular finito

En el capítulo ?? se presentaron las gramáticas de concatenación de rango y se revisaron antecedentes en los que el SAT se aborda mediante intersecciones entre lenguajes. En esos antecedentes, la fórmula booleana se transforma en un lenguaje regular que representa las interpretaciones verdaderas bajo el supuesto de que cada ocurrencia de variable es independiente. Después, una gramática impone la consistencia entre las ocurrencias de una misma variable. La satisfacibilidad se reduce entonces al problema de decidir si la intersección de ambos lenguajes es vacía [3, 1].

El paso de las gramáticas libres del contexto y de las gramáticas de concatenación de rango simples a las RCG no conserva de manera automática la tratabilidad de ese enfoque. Bertsch y Nederhof prueban que, para RCG, el vacío de la intersección con un lenguaje regular arbitrario es indecidible. También prueban que, si el lenguaje regular es finito y se representa mediante un autómata acíclico, el problema de decidir si la intersección es no vacía es NP-completo [2]. En el presente capítulo se formaliza ese último caso mediante un algoritmo explícito de enumeración de caminos aceptantes.

La formalización no sustituye el resultado de complejidad de Bertsch y Nederhof. Su objetivo es precisar qué procedimiento de decisión se obtiene cuando el autómata es acíclico, demostrar su correctitud y caracterizar su costo en función del número de caminos aceptantes del autómata.

El capítulo se organiza de la siguiente manera. En la sección 3.1 se presentan los lenguajes regulares finitos. En la sección 3.2 se formalizan los autómatas acíclicos y sus caminos aceptantes. En la sección 3.3 se define el problema de vacío de la intersección. En la sección 3.4 se presenta el algoritmo de decisión, se demuestra su correctitud y se analiza su complejidad. En la sección 3.5 se modela el SAT como un caso de ese problema. Por último, en la sección 3.6 se

lee el resultado en términos del SAT y se motiva la búsqueda de subclases no lineales con vacío tratable.

3.1. Lenguajes regulares finitos

El enfoque SAT por intersección trabaja con un lenguaje regular que representa interpretaciones de una fórmula. Para una fórmula con una cantidad finita de ocurrencias de variables, ese lenguaje es finito. La presente sección fija la noción de lenguaje regular finito que será usada en el resto del capítulo.

Definición 3.1. Un lenguaje L es **finito** si existe un entero $k \geq 0$ tal que L tiene exactamente k cadenas.

Definición 3.2. Un **lenguaje regular finito** es un lenguaje finito que también es regular.

Una familia de lenguajes finitos puede tener una representación mediante autómatas de tamaño polinomial y contener, al mismo tiempo, una cantidad exponencial de cadenas.

Por ejemplo, para cada $n \geq 1$, sea

$$L_n = \{0, 1\}^n.$$

El lenguaje L_n es finito y contiene 2^n cadenas. No obstante, existe un autómata finito con $n + 1$ estados que reconoce L_n : el autómata avanza de un estado al siguiente al leer 0 o 1, y acepta solo después de leer exactamente n símbolos.

El ejemplo anterior muestra por qué no basta con saber que el lenguaje regular es finito. La finitud garantiza que una enumeración termina, pero no garantiza que termine en tiempo polinomial respecto al tamaño de la representación. Cuando un lenguaje regular es finito, admite una representación mediante un autómata acíclico, que se define en la sección siguiente.

3.2. Autómatas acíclicos

En el presente capítulo se usa el autómata finito del capítulo ??, con una diferencia: la función de transición es parcial, no total, es decir, $\delta(q, a)$ puede no estar definida para algunos pares (q, a) . Cuando $\delta(q, a)$ no está definida, la lectura del símbolo a en el estado q no conduce a ningún estado.

Definición 3.3. Sean $p_0, p_1, \dots, p_m$ estados de Q y $a_1, a_2, \dots, a_m$ símbolos de Σ , con $m \geq 0$. Un **camino** desde p_0 hasta p_m es una secuencia

$$\pi = p_0 a_1 p_1 a_2 p_2 \cdots a_m p_m$$

tal que, para cada $1 \leq i \leq m$, se cumple

$$\delta(p_{i-1}, a_i) = p_i.$$

La cadena que se lee a lo largo de π , denotada $w(\pi)$, es $a_1 a_2 \cdots a_m$. Si $m = 0$, el camino no contiene transiciones y la cadena que se lee es ε .

Definición 3.4. Un camino π en el autómata es **aceptante** si empieza en el estado inicial q_0 y termina en un estado de aceptación. El lenguaje aceptado por A es

$$L(A) = \{w(\pi) \mid \pi \text{ es un camino aceptante de } A\}.$$

La aciclicidad se define a partir de los caminos del autómata.

Definición 3.5. Un autómata finito A es **acíclico** si ningún camino de A con al menos una transición empieza y termina en el mismo estado.

La aciclicidad es suficiente para asegurar que el lenguaje aceptado es finito. En los tres lemas siguientes se enuncian la cota de longitud de los caminos, que se usa en el análisis de complejidad, la finitud del lenguaje aceptado y la representación de todo lenguaje finito mediante un autómata acíclico.

Lema 3.1. Sea $A = (Q, \Sigma, \delta, q_0, F)$ un autómata finito acíclico. Todo camino de A tiene longitud a lo sumo $|Q| - 1$. En consecuencia, toda cadena aceptada por A tiene longitud a lo sumo $|Q| - 1$ [2].

Lema 3.2. Si A es un autómata finito acíclico, entonces $L(A)$ es finito [2].

Lema 3.3. Todo lenguaje finito puede reconocerse mediante un autómata finito acíclico [2].

En el análisis de complejidad del presente capítulo, el autómata forma parte de la entrada, de modo que el costo se expresa en función de su tamaño. Ese costo depende, ante todo, de la cantidad de caminos aceptantes del autómata. Por ello se introduce la siguiente notación.

Definición 3.6. Sea A un autómata finito acíclico. Se denota por $p(A)$ la cantidad de caminos aceptantes de A .

Como δ asigna a lo sumo un estado a cada par (q, a) , cada cadena aceptada determina un único camino aceptante. En consecuencia,

$$p(A) = |L(A)|.$$

3.3. Problema de vacío de la intersección

Las secciones anteriores describieron la parte regular de la entrada. El **problema del vacío de la intersección de una RCG con un lenguaje regular finito** toma como entrada una RCG G y un autómata finito acíclico A , y pregunta si $L(G) \cap L(A) \neq \emptyset$.

La intersección es no vacía si y solo si existe una cadena que se reconoce a la vez por la RCG G y por el autómata A :

$$L(G) \cap L(A) \neq \emptyset \iff \exists w \in \Sigma^* \text{ tal que } w \in L(G) \text{ y } w \in L(A).$$

El algoritmo de la sección siguiente decide el problema recorriendo las cadenas aceptadas por el autómata acíclico.

3.4. Algoritmo de decisión

El resultado de Bertsch y Nederhof garantiza que este caso es decidible, pero no proporciona un algoritmo explícito ni expresa su costo en función de la cantidad de cadenas aceptadas [2]. La presente sección da ese algoritmo y analiza su costo.

Como el autómata de entrada es acíclico, sus caminos aceptantes son finitos. Además, como δ asigna a lo sumo un estado a cada par (q, a) , esos caminos están en correspondencia uno a uno con las cadenas aceptadas. Por tanto, una forma directa de decidir si la intersección es no vacía consiste en recorrer las cadenas aceptadas por el autómata y aplicar, a cada una, un algoritmo de reconocimiento para RCG. El procedimiento es determinista y, en general, no se ejecuta en tiempo polinomial. Su utilidad está en exponer el parámetro responsable del costo: la cantidad de cadenas aceptadas por el autómata.

Se usará como subrutina un procedimiento de reconocimiento para RCG, que se denota

$$\text{RCG-RECOGNIZE}(G, w).$$

La subrutina devuelve TRUE si y solo si $w \in L(G)$. El reconocimiento de una cadena por una RCG se resuelve en tiempo polinomial en la longitud de la cadena [2, 4].

Antes de enumerar cadenas conviene eliminar los estados que no pueden participar en ningún camino aceptante. Esa reducción evita recorrer ramas que no contribuyen al lenguaje del autómata.

Definición 3.7. Sea $A = (Q, \Sigma, \delta, q_0, F)$ un autómata finito. Un estado $q \in Q$ es **útil** si es alcanzable desde q_0 y desde q se puede alcanzar algún estado de aceptación.

El conjunto de estados útiles se obtiene mediante dos recorridos del autómata. En el primero se determinan los estados alcanzables desde q_0 : se parte de $\{q_0\}$ y se agrega $\delta(q, a)$ por cada estado q ya incluido y cada símbolo a para el que $\delta(q, a)$ está definida, hasta que no se agregue ningún estado. En el segundo se determinan los estados desde los que se alcanza un estado de aceptación: se parte de F y se agrega cada estado q tal que $\delta(q, a)$ ya está incluido para algún símbolo a , hasta que no se agregue ninguno. Un estado es útil cuando pertenece a los dos conjuntos. El autómata $\text{USEFUL-PART}(A)$ tiene por estados los útiles, por estado inicial q_0 , por estados de aceptación los de F que son útiles y por transiciones las $\delta(q, a) = q'$ en las que q y q' son útiles. En cada recorrido, cada estado y cada transición se visitan a lo sumo una vez, de modo que la reducción se calcula en tiempo lineal en el tamaño del autómata.

El pseudocódigo de USEFUL-PART es el siguiente.

USEFUL-PART(A)

```

1   $R = \{q_0\}$ 
2  while there is a transition  $\delta(q, a) = q'$  with  $q \in R$  and  $q' \notin R$ 
3       $R = R \cup \{q'\}$ 
4   $C = F$ 
5  while there is a transition  $\delta(q, a) = q'$  with  $q' \in C$  and  $q \notin C$ 
6       $C = C \cup \{q\}$ 
7   $U = R \cap C$ 
8  return  $(U, \Sigma, \delta', q_0, F \cap U)$ , where  $\delta'$  is the restriction of  $\delta$  to  $U$ 

```

El autómata $\text{USEFUL-PART}(A)$ acepta el mismo lenguaje que A ; cuando no hay estados útiles, ese lenguaje es vacío.

El algoritmo principal se presenta a continuación.

NONEMPTY-INTERSECTION(G, A)

```

1   $A' = \text{USEFUL-PART}(A)$ 
2  if the initial state of  $A'$  does not exist
3      return FALSE
4  return SEARCH( $q_0, \varepsilon$ )

```

La subrutina SEARCH recorre las cadenas aceptadas por el autómata. El parámetro q indica el estado actual y w almacena la cadena que se lee a lo largo del camino desde q_0 hasta ese estado.

SEARCH(q, w)

```

1  if  $q \in F$  and RCG-RECOGNIZE( $G, w$ )
2      return TRUE
3  for each symbol  $a \in \Sigma$  such that  $\delta'(q, a)$  is defined
4       $r = \delta'(q, a)$ 
5      if SEARCH( $r, wa$ )
6          return TRUE
7  return FALSE

```

En el pseudocódigo, δ' denota la función de transición de A' . La línea 1 de SEARCH verifica la pertenencia a la RCG cada vez que el camino recorrido termina en un estado de aceptación. Si la verificación es positiva, la intersección es no vacía y el algoritmo termina. Si la verificación es negativa, el recorrido continúa, ya que desde un estado de aceptación pueden salir transiciones hacia otros caminos aceptantes. La línea 5 actualiza la cadena asociada al camino mediante la concatenación del símbolo leído.

La aciclicidad de A garantiza la terminación de la búsqueda. Los estados $q_0, q_1, \dots, q_d$ que aparecen en una secuencia de llamadas recursivas anidadas forman un camino de A' : en cada paso, de la llamada en el estado q se pasa a la llamada en el estado $\delta'(q, a)$. La profundidad d de la recursión es entonces el número de transiciones de ese camino. Como A' es acíclico, ese camino no

contiene dos veces el mismo estado, de modo que ningún estado se repite a lo largo de la recursión. Por el lema 3.1, $d \leq |Q| - 1$. Como en cada llamada se generan a lo sumo $|\Sigma|$ llamadas recursivas, el árbol de recursión es finito y la búsqueda termina.

3.4.1. Correctitud

La correctitud se demuestra separando tres hechos. El primero establece que toda cadena que se evalúa con RCG-RECOGNIZE pertenece al lenguaje del autómata. El segundo establece que toda cadena aceptada por el autómata se evalúa exactamente una vez. El tercero se apoya en la correctitud de RCG-RECOGNIZE.

Lema 3.4. *Toda cadena w para la que el algoritmo ejecuta RCG-RECOGNIZE(G, w) pertenece a $L(A)$.*

Demostración. La subrutina SEARCH ejecuta RCG-RECOGNIZE(G, w) solo en la línea 1, cuando el estado actual q pertenece a F . El valor de w se construye concatenando los símbolos que se leen desde q_0 hasta q . Por tanto, existe un camino desde q_0 hasta un estado de aceptación a lo largo del cual se lee w . Por la definición de aceptación, $w \in L(A')$. Como $A' = \text{USEFUL-PART}(A)$ acepta el mismo lenguaje que A , se tiene $L(A') = L(A)$. Luego $w \in L(A)$. $\square$

Lema 3.5. *Para toda cadena $w \in L(A)$, el algoritmo ejecuta RCG-RECOGNIZE(G, w) exactamente una vez.*

Demostración. Sea $w = a_1a_2 \cdots a_m$ una cadena de $L(A)$. Como para cada par (q, a) hay a lo sumo un estado $\delta(q, a)$, existe un único camino de A que comienza en q_0 y a lo largo del cual se lee w :

$$q_0a_1q_1a_2q_2 \cdots a_mq_m.$$

Además, como $w \in L(A)$, se tiene $q_m \in F$. Todos los estados de ese camino son útiles: cada uno es alcanzable desde q_0 por el prefijo correspondiente, y desde cada uno se alcanza un estado de aceptación por el sufijo correspondiente. Por tanto, el camino también pertenece a A' .

La subrutina SEARCH recorre, para cada estado actual, todos los símbolos a para los cuales $\delta'(q, a)$ está definida. Por inducción sobre la longitud de los prefijos de w , el algoritmo realiza una llamada recursiva asociada al único prefijo de longitud i de w , para cada $0 \leq i \leq m$. Al llegar al estado q_m , la cadena acumulada es w y se cumple $q_m \in F$. En ese momento se ejecuta RCG-RECOGNIZE(G, w).

La ejecución es única porque en A' hay a lo sumo un estado $\delta'(q, a)$ por cada par (q, a) : ninguna cadena se lee a lo largo de dos caminos distintos desde q_0 . Por tanto, el algoritmo no puede alcanzar un estado de aceptación con la misma cadena w por dos recorridos diferentes. $\square$

Lema 3.6. *Para toda cadena w , la subrutina RCG-RECOGNIZE(G, w) devuelve TRUE si y solo si $w \in L(G)$.*

Demostración. El reconocimiento de una cadena por una RCG se decide en tiempo polinomial [2, 4]; RCG-RECOGNIZE es un procedimiento que resuelve ese problema y devuelve TRUE si y solo si $w \in L(G)$. $\square$

Teorema 3.1. *El algoritmo NONEMPTY-INTERSECTION(G, A) devuelve TRUE si y solo si*

$$L(G) \cap L(A) \neq \emptyset.$$

Demostración. Supóngase primero que el algoritmo devuelve TRUE. Entonces, para alguna cadena w , se ejecutó RCG-RECOGNIZE(G, w) y la subrutina devolvió TRUE. Por el lema 3.4, $w \in L(A)$. Por el lema 3.6, $w \in L(G)$. Luego $w \in L(G) \cap L(A)$, y la intersección es no vacía.

Supóngase ahora que $L(G) \cap L(A) \neq \emptyset$. Entonces existe una cadena w tal que $w \in L(G)$ y $w \in L(A)$. Por el lema 3.5, el algoritmo ejecuta RCG-RECOGNIZE(G, w) exactamente una vez. Por el lema 3.6, esa ejecución devuelve TRUE. Por tanto, el algoritmo devuelve TRUE. $\square$

3.4.2. Complejidad

La complejidad del algoritmo depende de dos componentes. El primero es la enumeración de cadenas aceptadas por el autómata acíclico. El segundo es el reconocimiento de cada cadena mediante la RCG. En la presente subsección se expresan ambos costos de manera separada.

Sea $|A| = |Q| + |\delta|$ el tamaño del autómata, donde $|\delta|$ denota la cantidad de pares (q, a) para los cuales la transición está definida. Sea $p(A)$ el número de caminos aceptantes de A . Como se estableció en la sección 3.2, $p(A) = |L(A)|$.

Después de eliminar estados no útiles, todo camino recorrido por el algoritmo es prefijo de algún camino aceptante. Como cada camino aceptante tiene longitud a lo sumo $|Q| - 1$, el número de prefijos recorridos está acotado por

$$p(A) |Q|.$$

Por tanto, el costo de la enumeración, sin contar las llamadas a RCG-RECOGNIZE, queda acotado por

$$O(p(A) |Q|),$$

si se usan listas de adyacencia y la cadena acumulada se mantiene en una pila de símbolos.

Sea $R_G(n)$ el tiempo que demora RCG-RECOGNIZE(G, w) sobre una cadena w de longitud n . Para una RCG fija, $R_G(n)$ es polinomial en n . Si la gramática forma parte de la entrada, el costo es polinomial en la longitud de la cadena para cada gramática fijada, y el exponente depende de parámetros estructurales de la gramática, como la cantidad de variables que pueden recibir rangos en una cláusula [2, 4].

Por el lema 3.1, toda cadena aceptada tiene longitud a lo sumo $|Q| - 1$. Luego el costo total del algoritmo se acota por

$$O(|A| + p(A) |Q| + p(A) R_G(|Q| - 1)).$$

De forma equivalente,

$$O(|A| + p(A)(|Q| + R_G(|Q| - 1))).$$

La cota anterior es polinomial si $p(A)$ está acotado por un polinomio en $|A|$. Sin embargo, $p(A)$ no está polinomialmente acotado para todos los autómatas acíclicos. En la sección 3.1 ya se mostró un autómata así: el que reconoce $L_n = \{0, 1\}^n$, con $n + 1$ estados, acíclico y con 2^n cadenas aceptadas. Por la igualdad $p(A) = |L(A)|$, ese autómata tiene 2^n caminos aceptantes con un tamaño lineal en n .

Por tanto, la enumeración de cadenas aceptadas puede tener costo exponencial en el tamaño del autómata, incluso para autómatas acíclicos. En consecuencia, el algoritmo del presente capítulo no contradice la NP-completitud del caso finito con autómatas acíclicos demostrada en [2]. La enumeración determinista decide el problema, pero puede tardar tiempo exponencial.

La cota anterior permite obtener una condición suficiente de tratabilidad para clases restringidas de autómatas acíclicos.

Teorema 3.2. *Sea $\mathcal{A}$ una clase de autómatas finitos acíclicos tal que existe un polinomio r con*

$$p(A) \leq r(|A|)$$

para todo $A \in \mathcal{A}$. Entonces el problema del vacío de la intersección, restringido a autómatas de $\mathcal{A}$, se decide en tiempo polinomial para cada RCG fija.

Demostración. Por la cota del algoritmo,

$$O(|A| + p(A)(|Q| + R_G(|Q| - 1)))$$

es suficiente para decidir el problema. Si $A \in \mathcal{A}$, entonces $p(A) \leq r(|A|)$. Además, $|Q| \leq |A|$. Para una RCG fija, $R_G(|Q| - 1)$ es polinomial en $|Q|$, y por tanto en $|A|$. La composición de esos polinomios produce un tiempo polinomial en $|A|$. $\square$

El teorema anterior debe leerse como una condición sobre la representación regular. Si una clase de autómatas acíclicos tiene polinomialmente muchas cadenas aceptadas, la enumeración alcanza para obtener una decisión polinomial. Si una clase admite autómatas que representan exponencialmente muchas cadenas, el algoritmo puede ser exponencial.

Generar cada cadena aceptada en tiempo polinomial no es lo mismo que enumerarlas todas en tiempo polinomial. La cantidad de cadenas aceptadas puede ser exponencial, de modo que recorrerlas todas cuesta tiempo exponencial aun cuando cada una se obtenga en tiempo polinomial. Para decidir el vacío de la intersección hace falta, en el peor caso, recorrer todas: cuando la intersección es vacía, ninguna cadena aceptada pertenece a $L(G)$, y el recorrido solo concluye al agotarlas.

3.5. Modelación del SAT con RCG

El SAT se modela con el problema de vacío de las secciones anteriores. A una fórmula con n ocurrencias de variable se le asocia el autómata booleano del capítulo ??, finito y acíclico, que reconoce sus interpretaciones verdaderas bajo el supuesto de que cada ocurrencia es independiente. Queda imponer que las ocurrencias de una misma variable tomen el mismo valor. Tras SATCF y SATSRC, este trabajo analiza esa consistencia con una RCG.

Sea $\ell_1 \cdots \ell_n$ la secuencia de ocurrencias de la fórmula, con ℓ_i la variable de la ocurrencia i . La gramática de consistencia G_{cons} tiene por terminales $T = \{0, 1\}$, por variables $X, W_0, W_1, \dots, W_n$ y tres juegos de predicados, todos de aridad uno: el predicado inicial S , un predicado Eq_v por cada variable booleana v y los predicados $L_0, L_1, \dots, L_n$. El predicado L_j reconoce las cadenas de longitud j , mediante las cláusulas

$$L_j(cX) \rightarrow L_{j-1}(X) \quad (c \in \{0, 1\}, 1 \leq j \leq n), \quad L_0(\varepsilon) \rightarrow \varepsilon.$$

Para cada variable booleana v , con ocurrencias en las posiciones $i_1 < i_2 < \dots < i_r$, el predicado Eq_v reconoce las cadenas de longitud n en las que esas posiciones llevan el mismo símbolo. Sus cláusulas son dos, una por cada símbolo $b \in \{0, 1\}$:

$$Eq_v(W_0 b W_1 b \cdots b W_r) \rightarrow L_{g_0}(W_0) L_{g_1}(W_1) \cdots L_{g_r}(W_r).$$

El argumento es un patrón: el símbolo b ocupa las posiciones de v , y cada variable W_k cubre una subcadena: la anterior a la primera ocurrencia, la comprendida entre dos ocurrencias consecutivas o la posterior a la última. Esas subcadenas tienen longitudes $g_0 = i_1 - 1$, $g_k = i_{k+1} - i_k - 1$ y $g_r = n - i_r$, y cada una se reconoce por el predicado L_{g_k} correspondiente. Una vez fijada la fórmula, las posiciones i_k y las longitudes g_k son constantes, de modo que el conjunto de cláusulas es finito y cada cláusula tiene la forma de la definición ??.

La consistencia de la fórmula es la conjunción de esas restricciones, y se expresa con una cláusula no lineal con el mismo argumento en los m predicados Eq_v , donde m es la cantidad de variables booleanas de la fórmula:

$$S(X) \rightarrow Eq_{v_1}(X) Eq_{v_2}(X) \cdots Eq_{v_m}(X).$$

La variable X es la misma en los m predicados, así que las m restricciones se aplican a la misma cadena. Con esta cláusula queda completo el conjunto de cláusulas y, con él, la gramática G_{cons} . El lenguaje reconocido por G_{cons} se denota $L(G_{\text{cons}})$; el lema siguiente lo caracteriza.

Lema 3.7. *Sea G_{cons} la gramática de consistencia de una fórmula booleana con n ocurrencias de variable. El lenguaje $L(G_{\text{cons}})$ es el conjunto de las cadenas consistentes de la fórmula.*

Demostración. Primero se establece que el predicado L_j deriva en la cadena vacía sobre una cadena u si y solo si $|u| = j$. La prueba es por inducción sobre

j : la cláusula $L_0(\varepsilon) \rightarrow \varepsilon$ cubre el caso $j = 0$, y con cada cláusula $L_j(cX) \rightarrow L_{j-1}(X)$ se consume un símbolo.

Sea w una cadena consistente; como lleva un símbolo por ocurrencia, $|w| = n$. Para cada variable booleana v , sea b el símbolo común de sus posiciones en w . Entonces w se descompone como $u_0 b u_1 b \cdots b u_r$ con $|u_k| = g_k$: en el patrón del símbolo b , la sustitución de rango asocia cada variable W_k a la subcadena u_k , y cada predicado $L_{g_k}(u_k)$ deriva en la cadena vacía. Luego $Eq_v(w)$ deriva en la cadena vacía. Como esto vale para las m variables booleanas, con la cláusula de S se deriva en la cadena vacía y $w \in L(G_{\text{cons}})$.

Sea ahora $w \in L(G_{\text{cons}})$. La única cláusula de S es $S(X) \rightarrow Eq_{v_1}(X) \cdots Eq_{v_m}(X)$, de modo que cada $Eq_v(w)$ deriva en la cadena vacía. En esa derivación se aplica una de las dos cláusulas de Eq_v , con algún símbolo b : entonces $w = u_0 b u_1 b \cdots b u_r$, y cada predicado $L_{g_k}(u_k)$ deriva en la cadena vacía, así que $|u_k| = g_k$. Como $g_0 + g_1 + \cdots + g_r + r = n$ por la definición de las longitudes g_k , se tiene $|w| = n$, y las posiciones $i_1, \dots, i_r$ de w llevan el símbolo b . Como esto vale para cada variable booleana, w es consistente. $\square$

Por ejemplo, considere la fórmula $x_1 \wedge (x_2 \vee x_1)$ del capítulo ??, cuya secuencia de ocurrencias es $x_1 x_2 x_1$, con $n = 3$ y $m = 2$. La variable x_1 ocurre en las posiciones 1 y 3, con subcadenas de longitudes $g_0 = 0$, $g_1 = 1$ y $g_2 = 0$; la variable x_2 ocurre en la posición 2, con subcadenas de longitudes $g_0 = 1$ y $g_1 = 1$. Las cláusulas de G_{cons} son las siguientes, omitidas las de L_2 y L_3 , que en ningún patrón aparecen:

$$\begin{aligned} S(X) &\rightarrow Eq_{x_1}(X) Eq_{x_2}(X), \\ Eq_{x_1}(W_0 0 W_1 0 W_2) &\rightarrow L_0(W_0) L_1(W_1) L_0(W_2), \\ Eq_{x_1}(W_0 1 W_1 1 W_2) &\rightarrow L_0(W_0) L_1(W_1) L_0(W_2), \\ Eq_{x_2}(W_0 0 W_1) &\rightarrow L_1(W_0) L_1(W_1), \\ Eq_{x_2}(W_0 1 W_1) &\rightarrow L_1(W_0) L_1(W_1), \\ L_1(0X) &\rightarrow L_0(X), \quad L_1(1X) \rightarrow L_0(X), \quad L_0(\varepsilon) \rightarrow \varepsilon. \end{aligned}$$

La cadena 101 es consistente y se reconoce por G_{cons} mediante las derivaciones siguientes:

$$\begin{aligned} S(101) &\rightarrow Eq_{x_1}(101) Eq_{x_2}(101), \\ Eq_{x_1}(101) &\rightarrow L_0(\varepsilon) L_1(0) L_0(\varepsilon), \\ Eq_{x_2}(101) &\rightarrow L_1(1) L_1(1). \end{aligned}$$

A continuación se describen los pasos.

- **Primer paso:** en la cláusula $S(X) \rightarrow Eq_{x_1}(X) Eq_{x_2}(X)$, la sustitución de rango asocia la variable X al valor $X = 101$. De esta forma se deriva en $Eq_{x_1}(101) Eq_{x_2}(101)$.
- **Segundo paso:** en el patrón del símbolo 1 de Eq_{x_1} , la sustitución de rango asocia las variables W_0, W_1, W_2 a los valores $W_0 = \varepsilon, W_1 = 0$ y $W_2 = \varepsilon$. Con estos valores se deriva en $L_0(\varepsilon) L_1(0) L_0(\varepsilon)$.

- **Tercer paso:** en el patrón del símbolo 0 de Eq_{x_2} , la sustitución de rango asocia las variables W_0, W_1 a los valores $W_0 = 1$ y $W_1 = 1$. Con estos valores se deriva en $L_1(1) L_1(1)$.
- **Cuarto paso:** cada predicado L_1 deriva en $L_0(\varepsilon)$, y cada predicado $L_0(\varepsilon)$ deriva en la cadena vacía por la cláusula terminal, con lo que la cadena 101 queda reconocida y $101 \in L(G_{\text{cons}})$.

La cadena 110, con la que en el capítulo ?? se llega al estado V del autómata booleano, no es consistente. En $Eq_{x_1}(110)$ ningún patrón admite una sustitución de rango: el del símbolo 0 exige 0 en la posición 1, donde la cadena lleva 1, y el del símbolo 1 exige 1 en la posición 3, donde la cadena lleva 0. Como $Eq_{x_1}(110)$ no deriva en la cadena vacía, $110 \notin L(G_{\text{cons}})$.

La satisfacibilidad de la fórmula equivale a que la intersección de ambos lenguajes no sea vacía, como establece el teorema siguiente.

Teorema 3.3. *Sean G_{cons} la gramática de consistencia de una fórmula booleana y A su autómata booleano. La fórmula es satisfacible si y solo si $L(G_{\text{cons}}) \cap L(A) \neq \emptyset$.*

Demostración. El lenguaje del autómata booleano está formado por las cadenas que hacen verdadera la fórmula cuando cada ocurrencia se trata por separado [3].

A cada interpretación I le corresponde la cadena $w_I = b_1 b_2 \dots b_n$, donde b_i es el valor, 1 o 0, que I asigna a la variable ℓ_i . La cadena w_I es consistente, porque las posiciones de una misma variable llevan el valor que I asigna a esa variable, y toda cadena consistente es w_I para la interpretación I que asigna a cada variable el símbolo común de sus posiciones. Además, en w_I cada ocurrencia lleva el valor que I asigna a su variable, de modo que $w_I \in L(A)$ si y solo si I hace verdadera la fórmula.

Supóngase que la fórmula es satisfacible. Entonces alguna interpretación I la hace verdadera. La cadena w_I es consistente, de modo que $w_I \in L(G_{\text{cons}})$ por el lema 3.7, y $w_I \in L(A)$. Luego la intersección es no vacía.

Supóngase ahora que la intersección es no vacía. Entonces alguna cadena w pertenece a $L(G_{\text{cons}})$ y a $L(A)$. Por el lema 3.7, w es consistente, de modo que $w = w_I$ para la interpretación I que asigna a cada variable el símbolo común de sus posiciones. De $w_I \in L(A)$ se sigue que I hace verdadera la fórmula. Por tanto, la fórmula es satisfacible. $\square$

El SAT se reduce entonces al problema de vacío de la intersección que este capítulo resuelve.

3.6. Consecuencias para el SAT y subclases tratables

Con la enumeración de este capítulo se decide esa satisfacibilidad: se recorren las cadenas aceptadas por el autómata booleano y se aplica a cada una el

reconocimiento de la RCG de consistencia. Las dos partes de la instancia tienen costo polinomial. El autómata booleano tiene $O(n^2)$ estados [3], con n la cantidad de ocurrencias de variables booleanas en la fórmula. La gramática G_{cons} tiene aridad uno y tamaño $O(nm)$, contado en símbolos, con m la cantidad de variables booleanas distintas: dos cláusulas de longitud $O(n)$ por variable booleana, más las $O(n)$ cláusulas de los predicados L_j . El reconocimiento con G_{cons} es polinomial: cada cláusula admite a lo sumo una sustitución de rango para cada cadena, porque los rangos de las variables W_k quedan determinados por las longitudes g_k , y esa sustitución se comprueba en tiempo lineal; el reconocimiento de una cadena de longitud n cuesta entonces $R_{G_{\text{cons}}}(n) = O(nm)$.

Toda cadena aceptada por el autómata booleano tiene longitud n , de modo que cada llamada a RCG-RECOGNIZE cuesta $O(nm)$. Con $|Q| = O(n^2)$, la cota de la subsección 3.4.2 queda en

$$O(n^2 + p(A)(n^2 + nm)) = O(n^2 + p(A)n^2),$$

pues $m \leq n$. El factor dominante es $p(A)$, la cantidad de cadenas aceptadas por el autómata booleano, que para una fórmula con muchas interpretaciones verdaderas bajo el supuesto de independencia es del orden de 2^n . La dificultad no está entonces en el tamaño de la instancia ni en el reconocimiento, sino en el vacío: decidir si $L(G_{\text{cons}}) \cap L(A)$ es vacío es el SAT, y para una RCG con un autómata finito acíclico ese problema es NP-completo [2].

En SATCF y SATSRC ese costo no aparece, y la razón no es una enumeración más hábil. En esos enfoques el vacío se decide desde el formalismo: el de una gramática libre del contexto y el de una sRCG se deciden en tiempo polinomial, sin recorrer las interpretaciones [3, 1]. Para las RCG no hay ese procedimiento directo: con la enumeración el problema se decide, pero el costo es el número de interpretaciones, no una propiedad del formalismo.

La no-linealidad tiene dos efectos. Por un lado, con la igualdad entre las subcadenas en que aparece una misma variable se amplía la clase de fórmulas cuya consistencia se expresa, más allá del orden de concatenación de rango simple de las sRCG. Por otro, por esa misma igualdad el vacío de la intersección deja de ser polinomial.

Quedan entonces dos vías hacia una extensión tratable de SATSRC. En la primera se restringen las fórmulas: si el autómata booleano tiene una cantidad de caminos aceptantes acotada por un polinomio en n , la cota anterior es polinomial y el algoritmo de enumeración decide la satisfacibilidad en tiempo polinomial, por lo que ya esa propuesta queda cerrada en este capítulo.

En la segunda vía se restringe la gramática de consistencia a una subclase no lineal de las RCG, que debe conservar parte de la expresividad que la no-linealidad añade sobre las sRCG y admitir un procedimiento polinomial para el vacío de la intersección que no recorra las interpretaciones. Con esta vía también se resuelve solo una clase de fórmulas, aquellas cuyas cadenas consistentes se reconocen por una gramática de la subclase: igual que en SATCF y SATSRC, la clase de fórmulas queda delimitada por el formalismo de la gramática de consistencia. Para hallar esa subclase hace falta una representación simbólica

de la intersección, sobre la que estudiar con qué restricciones el vacío se vuelve tratable.

En el capítulo siguiente se construye esa representación: una gramática producto entre una RCG y un lenguaje regular. Con ella el problema general no se vuelve polinomial, pero se obtiene el objeto sobre el que estudiar las subclases no lineales con vacío polinomial.

Bibliografía

- [1] Manuel Aguilera López. Problema de la satisfacibilidad booleana de concatenación de rango simple. In *Jornada Científica Estudiantil MatCom 2016*, pages 1–5, La Habana, Cuba, 2016. Facultad de Matemática y Computación, Universidad de La Habana.
- [2] Eberhard Bertsch and Mark-Jan Nederhof. On the complexity of some extensions of RCG parsing. In *Proceedings of the Seventh International Workshop on Parsing Technologies (IWPT 2001)*, pages 66–77, Beijing, China, 2001.
- [3] Alina Fernández Arias. El problema de la satisfacibilidad booleana libre del contexto. un algoritmo polinomial. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2007.
- [4] Laura Kallmeyer. *Parsing Beyond Context-Free Grammars*. Cognitive Technologies. Springer, Berlin, Heidelberg, 2010.